PART 02–03 • PLATFORM & CI/CD ENGINE

GitHub Architecture & Actions Deep Dive

Repository model, Enterprise topology, and the complete Actions execution engine

---

TOPICS COVERED

- › GitHub Platform Architecture
- › Permissions & RBAC
- › GitHub Security Features
- › Secret Scanning & Push Protection
- › Workflow YAML anatomy
- › Jobs, Steps & Actions
- › Expressions & Contexts
- › Execution Flow (YAML → VM)
- › Organizations & Enterprise
- › Fork Model & Visibility
- › Dependabot & CodeQL
- › Advanced Security
- › Triggers & Events (all 30+)
- › Composite & Reusable Workflows
- › Concurrency & Matrices
- › OIDC Token Generation

PART 2 — GitHub Architecture

2.1 GitHub Platform Overview

GitHub is a developer platform built atop Git that provides hosting, collaboration, security, and automation infrastructure. At its core, GitHub is a multi-tenant SaaS platform serving millions of repositories, but it is also available as GitHub Enterprise Server (GHES) — an on-premise appliance — and GitHub Enterprise Cloud (GHEC) — a dedicated cloud environment with data residency options.

Deployment Models

Model	Where it runs	Data residency	Scale
GitHub.com	Anthropic's cloud (Azure)	US/EU (limited)	All users
GitHub Enterprise Cloud (GHEC)	GitHub-managed Azure	US, EU, AU options	Enterprise orgs
GitHub Enterprise Cloud with Data Residency	Region-specific Azure	Strict region isolation	Regulated industries
GitHub Enterprise Server (GHES)	Your datacenter/cloud	Full customer control	Air-gapped support
GitHub AE (deprecated)	Isolated Azure tenant	N/A (discontinued)	N/A

2.2 Repository Architecture

A GitHub repository is more than a Git repository. It wraps the Git object store with metadata, access control, CI/CD integration, issue tracking, discussions, wiki, releases, packages, and a web interface. Internally, GitHub uses a Git backend (Gitaly, as open-sourced by GitLab; GitHub uses its own proprietary equivalent) fronted by a Ruby on Rails web application and backed by MySQL, Redis, Elasticsearch, and various microservices.

Repository Visibility

Visibility	Who can see?	Who can fork?	Enterprise use
Public	Everyone (unauthenticated)	Anyone on GitHub.com	Open source projects
Private	Repo members + org admins	Only org members (if enabled)	Most internal work
Internal	All org members (GHEC/GHES)	Any org member	InnerSource

■ *Internal repositories are the key to InnerSource programs. They enable read access for the whole company without making code publicly visible, while maintaining fine-grained write access control.*

Fork Model

GitHub's fork model creates a full copy of a repository under a different owner, maintaining an upstream relationship. Key behaviors:

• Forks share the underlying object store with the upstream (network repository) — no extra disk space for shared objects
• Pull requests can be created from a fork back to the upstream — the standard open-source contribution model
• Private repository forks inherit the parent's visibility; they cannot be made public
• Enterprise admins can restrict forking to within the organization

■ *Fork network objects: when a private repo is deleted, its fork network may retain access to blobs. Enterprise security policy should prohibit forking of sensitive repositories to external orgs.*

2.3 Organizations, Teams, and Permissions

Permission Hierarchy

GitHub's permission model is hierarchical: Enterprise → Organization → Repository → Branch → Environment.

```
Enterprise
■■■ Organization A
■   ■■■ Team: platform-engineers
■   ■   ■■■ Members: alice, bob, carol
■   ■■■ Repository: api-service
■   ■   ■■■ Team: platform-engineers  → Admin
■   ■   ■■■ Team: developers          → Write
■   ■   ■■■ Team: security-review     → Read
■   ■■■ Environment: production
■       ■■■ Required reviewers: [alice, bob]
■       ■■■ Wait timer: 10 minutes
■■■ Organization B
    ■■■ ...
```

Repository Roles

Role	Read	Triage	Write	Maintain	Admin
View code	✓	✓	✓	✓	✓
Open issues/PRs	✓	✓	✓	✓	✓
Manage issues/PRs		✓	✓	✓	✓
Push to non-protected branches			✓	✓	✓
Manage branch protection				✓	✓
Manage repo settings					✓

2.4 GitHub Advanced Security (GHAS)

GitHub Advanced Security is a suite of security features available free on public repositories and as a paid add-on for private repositories on GHEC/GHES. It encompasses three main capabilities:

Code Scanning with CodeQL

CodeQL is a semantic code analysis engine that models code as a queryable database. It can find complex security vulnerabilities like SQL injection, XSS, path traversal, and deserialization flaws that simple pattern

matching misses.

```
# .github/workflows/codeql.yml
name: CodeQL Analysis
on:
  push:
    branches: [main, 'release/**']
  pull_request:
    branches: [main]
  schedule:
    - cron: '0 2 * * 1'  # Weekly scan

jobs:
  analyze:
    runs-on: ubuntu-latest
    permissions:
      security-events: write
      contents: read
    strategy:
      matrix:
        language: [javascript-typescript, python, java]
    steps:
      - uses: actions/checkout@v4
      - uses: github/codeql-action/init@v3
        with:
          languages: ${{ matrix.language }}
          queries: security-extended  # or security-and-quality
      - uses: github/codeql-action/autobuild@v3
      - uses: github/codeql-action/analyze@v3
        with:
          category: /language:${{ matrix.language }}
```

Secret Scanning

GitHub's secret scanning engine uses patterns from 200+ token providers to detect secrets committed to repositories. It supports two modes:

- **Push protection**: Blocks the push before the secret enters the repository (preferred — remediation at the source)
- **Retroactive scanning**: Scans entire repository history, alerting on secrets already committed

```
# Repository-level secret scanning configuration
# .github/secret_scanning.yml
paths-ignore:
  - 'tests/fixtures/**'
  - '**/*.test.js'
  - 'docs/**'
```

■ *Push protection is the only mechanism that prevents a secret from ever entering git history. Once committed, the secret must be treated as compromised and rotated IMMEDIATELY, even if the commit is rewritten — pack files may have been distributed.*

Dependabot

Dependabot performs three functions: security alerts, security updates (auto-PRs for vulnerable dependencies), and version updates (scheduled version bump PRs).

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: npm
    directory: /
    schedule:
      interval: weekly
      day: monday
      time: "09:00"
      timezone: "America/New_York"
    open-pull-requests-limit: 10
    groups:
      dev-dependencies:
        patterns: ["@types/*", "eslint*", "jest*"]
    ignore:
```

```
      - dependency-name: "lodash"
        versions: ["4.x"]  # Pin major version
    assignees:
      - platform-bot
    reviewers:
      - security-team
  - package-ecosystem: docker
    directory: /
    schedule:
      interval: weekly
```

2.5 Codespaces

GitHub Codespaces provides cloud-hosted development environments backed by Azure VMs. Each codespace runs a Docker container (or multi-container dev container) with VS Code (browser or desktop tunnel), preconfigured with tools, extensions, and secrets.

```
# .devcontainer/devcontainer.json
{
  "name": "API Service Dev Environment",
  "image": "mcr.microsoft.com/devcontainers/python:3.11",
  "features": {
    "ghcr.io/devcontainers/features/docker-in-docker:2": {},
    "ghcr.io/devcontainers/features/kubectl-helm-minikube:1": {},
    "ghcr.io/devcontainers/features/node:1": {"version": "20"}
  },
  "forwardPorts": [3000, 5432, 6379],
  "postCreateCommand": "make dev-setup",
  "secrets": {
    "DATABASE_URL": {},
    "GITHUB_TOKEN": {}
  },
  "customizations": {
    "vscode": {
      "extensions": ["ms-python.python", "hashicorp.terraform", "ms-kubernetes-tools.vscode-kubernetes-tools"
    }
  }
}
```

■ *Codespaces prebuilds (configured in repository settings) pre-warm containers with cloned repositories and run postCreateCommand ahead of time, reducing startup from 3-5 minutes to under 30 seconds.*

PART 3 — GitHub Actions Deep Dive

3.1 Workflow YAML Anatomy

A GitHub Actions workflow is a YAML file in .github/workflows/. Each workflow consists of triggers (on:), optional global settings, and one or more jobs.

```yaml
name: Production Deployment Pipeline

# ■■ Triggers ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
on:
  push:
    branches: [main]
    paths-ignore: ['docs/**', '*.md']
  pull_request:
    branches: [main]
    types: [opened, synchronize, reopened, ready_for_review]
  workflow_dispatch:
    inputs:
      environment:
        description: 'Target environment'
        required: true
        type: choice
        options: [staging, production]
      dry_run:
        description: 'Dry run (no actual deployment)'
        type: boolean
        default: false

# ■■ Global settings ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: ${{ github.ref != 'refs/heads/main' }}

permissions:
  contents: read
  id-token: write   # Required for OIDC
  packages: write

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository }}
  PYTHON_VERSION: "3.11"

# ■■ Jobs ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${{ env.PYTHON_VERSION }}
          cache: pip
      - run: pip install -r requirements-dev.txt
      - run: ruff check .
      - run: pytest --cov=src --cov-report=xml
      - uses: codecov/codecov-action@v4

  build-image:
    needs: lint-and-test
    runs-on: ubuntu-latest
    outputs:
      image-digest: ${{ steps.build.outputs.digest }}
      image-ref: ${{ steps.meta.outputs.tags }}
    steps:
      - uses: actions/checkout@v4
      - uses: docker/metadata-action@v5
        id: meta
```

```
        with:
          images: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}
          tags: |
            type=sha,prefix={{branch}}-
            type=ref,event=tag
      - uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - uses: docker/build-push-action@v5
        id: build
        with:
          push: true
          tags: ${{ steps.meta.outputs.tags }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

  deploy-production:
    needs: build-image
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://api.example.com
    steps:
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789:role/github-deploy
          aws-region: us-east-1
      - run: |
          aws ecs update-service --cluster prod --service api \
            --force-new-deployment
```

3.2 Events — Complete Reference

Code Events

Event	Trigger	Key filters	Enterprise use
push	Commit pushed	branches, tags, paths	CI on every commit
pull_request	PR activity	types, branches	CI gate before merge
pull_request_target	PR from fork	Same as PR	Trusted CI for forks (DANGEROUS)
create	Branch/tag created	—	Notify on new features
delete	Branch/tag deleted	—	Cleanup stale environments
merge_group	Merge queue batch	branches	CI in merge queue

Workflow Control Events

Event	Trigger	Key use
workflow_dispatch	Manual trigger via UI/API	Manual deployments, debugging
repository_dispatch	POST to GitHub API	External system triggers
workflow_call	Called by another workflow	Reusable workflow entry point
workflow_run	Another workflow completes	Notification, cleanup after CI
schedule	CRON expression	Nightly builds, security scans

Security and Release Events

Event	Trigger
release	Release published/edited/created
deployment	Deployment created via API
deployment_status	Deployment status updated
check_suite / check_run	Check lifecycle events
security_advisory	GitHub security advisory activity
dependabot_alert	Dependabot alert activity

■ *pull_request_target runs in the context of the BASE branch, not the fork. It has access to secrets and is trusted — attackers can use it to exfiltrate secrets if you check out PR code and run it. Never use actions/checkout in pull_request_target without restricting to trusted actors.*

3.3 Expressions and Contexts

GitHub Actions expressions use ${{ }} syntax and are evaluated at runtime. They provide access to contexts (structured data about the workflow run) and built-in functions.

```
# Context hierarchy:
# github  - repository, event, actor, ref, sha, etc.
# env     - workflow/job/step environment variables
# vars    - repository/organization/enterprise variables
# secrets - encrypted secrets
# needs   - outputs from dependent jobs
# steps   - outputs from previous steps in same job
# runner  - OS, temp directory, workspace
# inputs  - workflow_dispatch inputs

# Common expressions:
${{ github.actor }}                    # User who triggered
${{ github.event_name }}               # Event type
${{ github.ref }}                      # refs/heads/main
${{ github.ref_name }}                 # main
${{ github.sha }}                      # Full commit SHA
${{ github.run_id }}                   # Unique run ID
${{ github.event.pull_request.number }}  # PR number

# Conditionals on jobs and steps:
if: github.ref == 'refs/heads/main'
if: github.event_name == 'pull_request'
if: contains(github.event.pull_request.labels.*.name, 'deploy')
if: failure()           # Run only if previous step failed
if: always()            # Run even if workflow is cancelled
if: success() || failure()  # Equivalent to always() except cancelled
if: cancelled()

# Functions:
${{ format('Hello {0}!', github.actor) }}
${{ join(matrix.os, ', ') }}
${{ toJSON(github) }}
${{ fromJSON(steps.meta.outputs.json).version }}
${{ contains('refs/heads/main', 'main') }}
${{ startsWith(github.ref, 'refs/tags/') }}
${{ hashFiles('**/package-lock.json') }}
```

3.4 Concurrency Control

Concurrency groups prevent multiple workflow runs from conflicting over shared resources (environments, infrastructure). They can either queue new runs or cancel in-progress ones.

```
# Cancel in-progress runs on PR pushes (save runner minutes):
concurrency:
  group: pr-${{ github.event.pull_request.number }}
  cancel-in-progress: true

# Queue deployments — never cancel production deploys:
concurrency:
  group: deploy-${{ github.event.inputs.environment }}
  cancel-in-progress: false

# Per-branch isolation:
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: ${{ github.ref != 'refs/heads/main' }}
  # → Cancel on feature branches, queue on main
```

3.5 Matrix Strategies

Matrix strategies run a job multiple times with different variable combinations, enabling cross-platform testing and parameterized pipelines.

```
jobs:
  test:
    strategy:
      fail-fast: false  # Don't cancel other matrix jobs on one failure
      max-parallel: 6   # Limit concurrent jobs (cost control)
      matrix:
        os: [ubuntu-22.04, windows-2022, macos-14]
        node: [18, 20, 22]
        include:
          # Add extra variables for specific combinations:
          - os: ubuntu-22.04
            node: 20
            run-coverage: true
        exclude:
          # Skip known bad combination:
          - os: windows-2022
            node: 18
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
      - run: npm test
      - if: matrix.run-coverage
        run: npm run coverage

  # Dynamic matrix from a previous job:
  dynamic-deploy:
    needs: discover-environments
    strategy:
      matrix:
        environment: ${{ fromJSON(needs.discover-environments.outputs.envs) }}
    environment: ${{ matrix.environment }}
    steps:
      - run: echo "Deploying to ${{ matrix.environment }}"

  discover-environments:
    outputs:
      envs: ${{ steps.list.outputs.matrix }}
    steps:
      - id: list
        run: |
          echo 'matrix=["staging","canary","production"]' >> $GITHUB_OUTPUT
```

3.6 Composite Actions and Reusable Workflows

Composite Actions

A composite action bundles multiple steps into a single reusable action, callable from any workflow with uses: org/repo/path@version.

```
# .github/actions/setup-python-env/action.yml
name: 'Setup Python Environment'
description: 'Install Python, dependencies, and configure caching'
inputs:
  python-version:
    description: 'Python version'
    required: false
    default: '3.11'
  install-dev:
    description: 'Install dev dependencies'
    required: false
    default: 'false'
outputs:
  cache-hit:
    description: 'Whether the cache was hit'
    value: ${{ steps.cache.outputs.cache-hit }}
runs:
  using: composite
  steps:
    - uses: actions/setup-python@v5
      with:
        python-version: ${{ inputs.python-version }}
    - name: Cache pip
      id: cache
      uses: actions/cache@v4
      with:
        path: ~/.cache/pip
        key: pip-${{ runner.os }}-${{ inputs.python-version }}-${{ hashFiles('**/requirements*.txt') }}
    - name: Install dependencies
      shell: bash
      run: |
        pip install -r requirements.txt
        if [ "${{ inputs.install-dev }}" = "true" ]; then
          pip install -r requirements-dev.txt
        fi
```

Reusable Workflows

Reusable workflows share entire job definitions (with runners, services, environment gates) across repositories — not just steps.

```
# .github/workflows/deploy-reusable.yml (in platform-team/workflows repo)
name: Reusable Deploy
on:
  workflow_call:
    inputs:
      environment:
        required: true
        type: string
      image-tag:
        required: true
        type: string
    secrets:
      DEPLOY_TOKEN:
        required: true
    outputs:
      deployment-url:
        description: "URL of deployed service"
        value: ${{ jobs.deploy.outputs.url }}

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: ${{ inputs.environment }}
    outputs:
      url: ${{ steps.deploy.outputs.url }}
```

```
    steps:
      - id: deploy
        run: |
          echo "url=https://${{ inputs.environment }}.example.com" >> $GITHUB_OUTPUT

# ■■ Caller workflow ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# In any team's repository:
jobs:
  deploy:
    uses: platform-team/workflows/.github/workflows/deploy-reusable.yml@v1
    with:
      environment: production
      image-tag: ${{ needs.build.outputs.tag }}
    secrets:
      DEPLOY_TOKEN: ${{ secrets.DEPLOY_TOKEN }}
```

■ *Reusable workflows support inheriting secrets with 'secrets: inherit' — useful for organization-level secrets that should flow to all callers without explicit passing.*

3.7 OIDC Token Generation and Cloud Authentication

GitHub Actions supports OIDC (OpenID Connect) to authenticate with cloud providers without storing long-lived credentials as secrets. The Actions runner contacts GitHub's OIDC provider to receive a signed JWT, which the cloud provider exchanges for short-lived credentials.

```
# AWS OIDC Setup (one-time IAM configuration):
# Trust policy for IAM role:
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Federated": "arn:aws:iam::123456789:oidc-provider/token.actions.githubusercontent.com"
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
        "token.actions.githubusercontent.com:sub": "repo:myorg/myrepo:environment:production"
      }
    }
  }]
}

# Workflow using OIDC:
jobs:
  deploy:
    permissions:
      id-token: write   # REQUIRED to request OIDC token
      contents: read
    steps:
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789:role/github-prod-deploy
          aws-region: us-east-1
          # No credentials stored anywhere!

      # Azure:
      - uses: azure/login@v2
        with:
          client-id: ${{ vars.AZURE_CLIENT_ID }}
          tenant-id: ${{ vars.AZURE_TENANT_ID }}
          subscription-id: ${{ vars.AZURE_SUBSCRIPTION_ID }}

      # GCP:
      - uses: google-github-actions/auth@v2
        with:
          workload_identity_provider: projects/123/locations/global/workloadIdentityPools/github/providers/gi
```

```
          service_account: deploy@project.iam.gserviceaccount.com
```

■ *Always use the most restrictive OIDC claim possible in your trust policy. Restrict by repo AND environment (not just repo) to prevent any workflow in the repo from assuming production roles.*

Interview Questions — GitHub & Actions

Q: What is the difference between pull_request and pull_request_target events?

A: pull_request runs in the context of the head (fork) branch with limited permissions and no secret access. pull_request_target runs in the context of the base branch with full secret access. The latter is dangerous if you checkout and run fork code, as an attacker can exfiltrate secrets.

Q: Explain the OIDC authentication flow for GitHub Actions to AWS.

A: The runner requests a JWT from GitHub's OIDC endpoint (token.actions.githubusercontent.com). The JWT contains claims like the repo, ref, environment, and actor. AWS STS verifies the JWT signature against GitHub's public key, checks the trust policy conditions, and issues temporary credentials via AssumeRoleWithWebIdentity. No secrets are stored in GitHub.

Q: When would you use a composite action vs a reusable workflow?

A: Use composite actions for reusable steps within a job (setup, build steps, notifications). Use reusable workflows for entire pipelines that need their own runner, services, environment gates, or approval requirements. Reusable workflows are the enterprise standard for shared deployment pipelines.

Q: How does concurrency: cancel-in-progress affect different branch types?

A: For feature branches and PRs, cancelling in-progress is desirable to save runner minutes on superseded pushes. For main/release branches, you want to queue (cancel-in-progress: false) to ensure every commit is tested/deployed. A common pattern: cancel-in-progress: ${{ github.ref != 'refs/heads/main' }}.